

A FOUNDER'S NOTES

Notes From a Broke Entrepreneur

A Guide, More or Less

Taj Nathan Donaldson

Founder, Totality

Notes From a Broke Entrepreneur: A Guide, More or Less

Copyright © 2026 Taj Nathan Donaldson. All rights reserved.

This PDF is licensed for the personal use of the buyer only. No part of this document may be reproduced, redistributed, resold, or shared without written permission from the author.

This book reflects one founder's real experience building Totality, a fitness coaching platform. It is not legal, financial, tax, or investment advice. Where legal or financial topics come up, consult a licensed professional about your own situation.

Company names, tools, and figures referenced throughout are used to describe real decisions made while building this business, as accurately as the author could recall and document them.

Contents

Nine chapters, plus a checklist

- 1** The Pitch That Wasn't a Pitch
- 2** Not Another All in One
- 3** The Money That Had a Deadline Attached
- 4** The Unglamorous Paperwork
- 5** Learning to Speak the Language
- 6** Nobody's Watching Your Work But You
- 7** Automations, and Filtering the Advice
- 8** Eighty Hours a Week, on Purpose
- 9** The Checklist

CHAPTER 1

The Pitch That Wasn't a Pitch

I asked my boss a simple question. Would you ever sell programs online? He thought about it for a second and said nup, but he wouldn't be against it if I wanted to have a go. I said okay. That was the whole conversation. No handshake, no excitement, nothing that felt like the start of anything. Just two people at a gym, one of them going back to whatever he was doing before I interrupted him.

A few hours later I sat down and started planning it properly. Not just planning, building. I opened up a page and started putting together a small website for it. Building websites wasn't something I actually knew how to do. A few weeks earlier I'd built him one for the gym itself, the gym he owned and I worked at, because he needed one and I offered, despite having no real idea what I was doing. Saying yes to things I didn't know how to do yet was already a habit of mine before any of this started.

I was a few hours into this second website when I noticed something that had nothing to do with the website itself. I'd built him one just the week before. Now here I was building him another one, separate from that, separate from everything. Meanwhile the gym was already running on Trainerize for programming and Mindbody for bookings and billing, plus whatever else had quietly been added over the years to cover nutrition tracking, client records, reminders, the stuff that doesn't show up on a tour but eats an afternoon every week. I sat back from the screen and thought about it properly for the first time. I wasn't fixing anything for him. I was adding to a pile that was already too tall. Another login. Another one-off. Another tool that didn't know the other tools existed.

That was the actual moment the real problem showed itself, and it didn't happen in a conversation with anyone. It happened alone, mid build, when I caught myself becoming exactly the thing I would eventually spend the next year trying to get rid of. The problem was never how do I sell more programs online. It was why does running a gym require this many disconnected tools that don't share a single piece of information with each other, and I only saw it because I was standing inside it with my own two hands.

I closed the website project and didn't go back to it. If the actual problem was too many disconnected tools, then a nicer looking website for selling programs online wasn't a solution to anything, it was just one more disconnected tool, the same mistake dressed up better. What I decided to build instead was a pitch, not for a website, for one platform that replaced everything the gym was already juggling. For the next four days I built that pitch. I wanted it to hold up on numbers, not enthusiasm, so I went looking for real ones: the size of the global fitness app market, its growth rate over the next decade, the size of the personal training market specifically, how many gyms exist in Australia alone, what a gym this size was realistically paying every month across five or six separate subscriptions and what that came to over a year. Thirty pages by the time I was done. I wasn't trying to make it sound exciting. I was trying to make the cost of the current setup impossible to wave away, because a feeling is easy to dismiss and a number on a slide in front of you is not.

Four days after that first short conversation, I went and found him and asked if he had time for a meeting right now, just fifteen minutes. He said yes. We sat down, I opened the laptop, and I walked him through it slide by slide, the problem first, then the cost of it in real dollars, then what I wanted to build instead.

Fifteen minutes turned into sixty. He was fully in it, asking questions, working through the numbers with me out loud, turning the idea over from angles I hadn't thought about yet. Eventually he had to go, he

had his daughter's soccer game to get to, and I figured that was the meeting done. Then my phone rang. It was him, calling from the car on the way there, wanting to keep going. We stayed on that call for another forty minutes while he drove to watch his kid play soccer. He told me straight out he was blown away, not by me, by the problem once it was laid out properly, by the numbers behind it, by the shape of what I was proposing to build. A hundred minutes total, spread across a fifteen minute ask and a phone call from a moving car, because the thing underneath it was real enough to hold his attention that long.

What I didn't tell him that day was that I had no idea how to build an app. I knew it myself, clearly, sitting right there while he was this excited. I had a degree in games, not software engineering, and nothing in my background actually told me whether a platform that was genuinely all in one, training and nutrition and the entire business side of a gym actually connected and talking to each other, was something one person could build. I said eighteen months anyway, shook on it in spirit if not in fact, and carried that gap privately for the whole build. I didn't tell him the truth until months later, after I put the finished app in his hands live on the Play Store, when I finally admitted out loud that I hadn't known if any of it was possible when I promised him eighteen months. What got me from one end of that gap to the other wasn't knowledge I already had. It was a stubbornness that runs close to obsessive, the same trait that used to have me rebuilding menus in a game engine at two in the morning because a transition felt half a frame off.

I beat that eighteen month deadline by more than a year, in five months, and I think that says less about how fast I work and more about what happens once you're building the right thing instead of the smaller, safer thing you already said yes to that morning.

I still have every version of that presentation. Twelve files, going back to when the whole thing was still called Live Well and hadn't become Totality yet. The pricing changed three times in the first few

months alone and has changed again more than once since, with more tiers now than I started with, because a solo trainer, a boutique studio, and a multi location gym were never going to fit under one price. None of that embarrasses me looking back at it. If your pricing hasn't changed since your first pitch, you probably haven't talked to enough real customers yet.

The lesson in this chapter isn't really about the fifteen minute meeting, even though that's the part that feels like the turning point. The actual turning point happened four days earlier, alone, mid build, noticing a pattern in my own hands that I hadn't gone looking for. Pay attention there. The meeting only worked because I showed up with something built on a real observation instead of a hunch I was hoping would land. Go find the observation first. Worry about whether you know how to build it after.

CHAPTER 2

Not Another All in One

Something needs saying before I get into how the actual differentiation got decided, because it explains why I was even in a position to try.

Fitness and software don't overlap much as industries. Most people who spend their careers coaching clients on a gym floor never go near a codebase, and most people who spend their careers writing software have never had to run a booking calendar, manage a roster of clients, and keep a business afloat at the same time. I've watched both sides try to solve this problem before me. Trainers build spreadsheets that fall apart under real use. Developers build training apps that get the actual coaching logic wrong, because they've never taken someone through a real mesocycle and watched what happens when the plan meets a tired, sore, inconsistent human being. The gap between the two worlds is where most of these attempts quietly die.

I happened to sit in both. I'm a personal trainer, but I also hold a bachelor's degree and a diploma, in technology and animation, from years before any of this started. For a long time I never connected the two halves on purpose. Looking back, I think I'd been circling this exact problem for years without ever naming it, half formed in the back of my head, because I had the coaching half of the picture from years on a gym floor and the technical half from my study, and neither half on its own was enough to see the whole shape of what was missing.

That overlap is the actual reason I could build this as cheaply as I did, by myself. I didn't need to hire a developer to translate what a coach needs into code, because I already knew what a coach needs. I didn't

need to hire a coach to sanity check whether the training logic held up, because that had been my job for years already. I could be the whole team because I already carried both halves of the team in one person. If there's a lesson in that for you rather than just backstory about me, it's this: look honestly at what your idea actually needs, and figure out which parts of that you already know cold and which parts you don't. Whatever's missing is the specific gap to go fill, by learning it or by bringing in someone who already has it. Not by copying my exact background, since you don't have it, and I don't have yours either.

Here's the problem with "let's build one app that does everything," once you're standing in that spot. Everyone's already had that idea. Every gym owner who's ever paid for five different subscriptions has muttered some version of it under their breath. The idea isn't rare. What's rare is doing it without ending up with a worse, shallower version of five separate tools duct-taped together, which is what most attempts at "all in one" actually are. Knowing I had a real problem to solve didn't mean I'd found something to build yet. It meant I had permission to go look properly.

So I did the boring thing. I went through the tools that were already out there, one at a time, and wrote down exactly what each one actually did well. Trainerize delivers programmes. That's it, that's the whole job, and it does that job fine. RP Strength has genuinely smart training logic behind it, real auto-regulation thinking, but no layer for a coach to oversee a roster of clients and no business tools underneath it at all. MacroFactor tracks nutrition properly, adaptive targets based on real weight trend data, but it lives completely on its own, disconnected from any training plan a client might be following. Mindbody runs the business side, bookings, billing, memberships, and does none of the actual coaching. Centr is closer to a content library than a coaching relationship. Every single one of them owned one slice and stopped there. Nobody had built the thing that sat across all of it, connecting a client's actual training data to their nutrition targets to the business running underneath both. That gap wasn't a

hunch. It was a list I made by actually comparing the tools feature by feature until the gap was undeniable on paper, not just something I felt in my gut.

The other place uniqueness actually got decided, rather than just claimed, was a smaller and less exciting decision than it probably should have been. I had a feature I wanted, an engine that watches how a client performs set by set and tells the coach when to adjust the programme. The obvious move in this market, the thing that gets you taken seriously in a pitch deck, is to call that AI. Everyone was doing it, still is. I looked at what I actually needed the engine to do and realized I didn't need a model guessing at a pattern. I needed it to compare a client's logged reps and RPE against what was expected at that point in their programme, using rules I could write down and defend, and flag the coach with a specific recommendation when the numbers said so. Deterministic. Explainable. No API call, no per-request cost, no chance of it hallucinating a recommendation I couldn't account for. I built it as logic instead of AI, on purpose, and it turned into one of the things people point at as proof Totality is different, not despite skipping the AI label but because of what skipping it actually meant: something a coach could trust completely because every recommendation traces back to a rule, not a guess.

I want to be honest about why that mattered so much to me at the time, beyond the technical argument. I didn't have money for AI API costs at scale, and I didn't fully trust a model I couldn't explain to a client's face if a recommendation went wrong. The deterministic choice was partly principle and partly just being a broke solo founder who needed something he could stand behind completely. Both things were true at once, and I've stopped pretending the decision was purely strategic. Sometimes the constraint you're working under and the differentiator you end up with are the same thing wearing two different hats.

I also built myself a piece of insurance I didn't tell anyone about for a while, which was a version of the pitch that assumed total failure. If Totality never sold a single subscription to anyone outside the one gym, it would still be replacing somewhere between five hundred and eight hundred dollars a month of tools that gym was already paying for. I wasn't pitching that as the goal. I was pitching it to myself as the floor, the version of this where I still came out ahead even in the worst case I could imagine. That number did more to keep me building through the harder months than any of the upside projections did, because upside is a guess and a floor is a fact you can check.

None of this came from having an idea nobody else had thought of. Plenty of people have thought "what if one app did all of it." The actual work was standing in the one place where I could see both halves of the problem at once, refusing to stop at the obvious version of the idea, and being honest enough with myself about my own constraints to turn a limitation into a decision I could defend. If you're trying to find what makes your idea different, that's the actual exercise. Not a brainstorm. A comparison table, filled in honestly, until the gap stops being a feeling and becomes a list.

CHAPTER 3

The Money That Had a Deadline Attached

People ask this question like it only has two answers. Did you bootstrap it, or did you take investment. As if those are two separate roads and you pick one at the start and stay on it. Mine didn't work like that, and I suspect most real ones don't.

Nick, my co-owner, the same one from that fifteen minute meeting that turned into a hundred, put in seven thousand dollars over six months. That's not a lump sum sitting in an account. It came in over time, closer to a wage that let me keep building without needing to pick up extra PT hours just to cover rent while the app was nowhere near making money. It also wasn't unconditional. We agreed that if I had nothing real to show him, a basic login, something that actually worked, by the end of August, the money stopped and I kept building on my own from there.

I want to be clear about what that condition actually was, because I think it's the most useful part of this chapter. It wasn't Nick being cautious or hedging his bet, at least not only that. I was the one who wanted a deadline attached to the money. Left to my own devices I will work on something forever without a real checkpoint forcing me to prove it's alive. Taking money with a condition built into it gave me exactly the kind of pressure I already knew I needed, from someone whose good opinion actually mattered to me. If you're the kind of person who needs outside accountability to keep momentum, and I very much am, tying a real deadline to real money someone else

is putting in is one of the more effective ways to manufacture that pressure honestly, instead of just telling yourself you'll be disciplined this time.

I didn't just clear that bar. By the time end of August rolled around I didn't have a basic login to show him, I had the entire app finished and live on the Play Store, about five months after that original meeting instead of the eighteen I'd promised. That's the same five months I mentioned back in Chapter 1. The deadline Nick and I set together turned out to be far more conservative than the one I'd already set for myself without telling him, and I hit both of them well inside the time either of us expected.

Once that six months of support was done, everything since has been on me. As of writing this, I've put roughly two thousand dollars of my own money into the tools that keep this running, and I want to be precise about what that money actually bought, because I think it's the single most useful thing in this chapter for you specifically. A large part of it went to Claude, which did an enormous amount of the actual database and code work. It never once supplied an idea. Every decision, every feature, every small detail of how something should look, feel, or behave, was argued out and decided by me first. The tool can move fast once you know exactly what you want built, but it doesn't know what your business needs unless you already know that yourself and can say it clearly. Hand over the deciding along with the building and what comes out won't be good, because nobody was actually driving it. That distinction between who builds and who decides is, I think, the most important thing in this entire book for anyone planning to use these tools the way I did.

The other tool in that two thousand dollars is HeyGen, and it sits in a completely different category. That one is purely for our ads, the marketing videos that go out to bring people in. Nothing it touches ever ends up inside the app itself. I keep those two separate on purpose, because it would be easy for someone reading this to assume the whole product got assembled by AI tools end to end, and that's not

what happened. The product got built by me, with a tool doing the typing. The ads got made with a different tool doing something closer to acting.

None of that spending has been clean or efficient. I've made plenty of expensive mistakes figuring out what's worth paying for and what isn't, and I'll get into some of the specific ones later in this book, the kind that cost real time and real money to catch. But the spending itself, coming entirely out of my own pocket since that six month window closed, is the part that actually makes this bootstrapped in the way people usually mean when they use that word.

So if you're trying to figure out which category your own business falls into, bootstrap or investment, I'd gently push back on the question itself. Mine is both, in sequence. Early, small, conditional support from someone close to the business, followed by a much longer stretch of self-funding once that support ran its course. I'd guess that's closer to the normal shape than either pure story people tell. The founders who say they bootstrapped the whole thing usually had some kind of early runway from somewhere, even if it was just a partner covering rent while they worked unpaid. The founders who took real investment are almost always still spending their own money on something the round didn't cover.

The part actually worth copying isn't which bucket the money came from. It's whether you attach a real, checkable condition to it, whoever it comes from, including yourself. Money without a deadline just becomes runway you can drift through. Money with a specific, honest checkpoint attached to it becomes pressure you can use on purpose.

CHAPTER 4

The Unglamorous Paperwork

I need to say something plainly before this chapter starts. I'm not a lawyer, I'm not an accountant, and nothing here is advice for your specific situation. This is what happened to me, written down so you go in with your eyes open, not a template to copy.

Here's how business smart I was at the start of this. I thought we needed an ABN and that was more or less the whole checklist in my head. That's it. I wasn't thinking about company structures, directorship, trusts, any of it, because I genuinely didn't know those things existed as decisions you had to make on purpose. I'd built a working app. I assumed the business side would be roughly that simple too.

The thing that saved me from my own blind spot wasn't a course I took or an article I read. It was Nick. He already ran a successful gym, which meant he already had accountants and already understood the shape of things I'd never had to think about. He's the one who sat with me and explained what a trust actually is, why it might matter for us, and what our accountants were talking about when they used terms I was nodding along to without really following. I didn't have to go learn all of this cold, alone, from nothing, because I had a co-owner who already carried that half of the knowledge, the same way I carried the technical half he didn't have. It's the same overlap I talked about back in Chapter 2, just showing up in a completely different part of the business.

The lawyer came from an even more personal source than I expected. My stepmum found him for us, the same lawyer she'd used for her own business, someone she already trusted and had a real relationship

with. We weren't googling "startup lawyer near me" and hoping for the best. We were using someone who came recommended by a person in my life who'd already been through something similar.

There's also a layer of even more basic admin sitting underneath all of this that's easy to overlook because it doesn't feel like "setting up a business," it just feels like errands. Something as simple as opening a proper business bank account, kept separate from your own personal money from day one, sounds like a five minute task until you're actually the one doing it, and you realize how many other pieces need to exist first, the entity structure, the ABN, sometimes a trust deed, before a bank will even set the account up properly. None of it is individually difficult. It's just slower than you expect at every single step, and the delays stack on top of each other. If I'd understood upfront how long the full chain actually takes, structure decided, accountant engaged, lawyer engaged, ABN sorted, bank account finally open, I would have started every part of it earlier instead of assuming I could knock it out in a spare afternoon whenever I got around to it.

I want to correct something I assumed going into this chapter, because I think the real version is more useful than the version I first imagined. I pictured the conversation about ownership and what happens if one of us leaves as the hard part, the thing people avoid because it feels like planning for a breakup before the relationship's even started. That's not how it went for us. Neither of us found it uncomfortable, because thinking several steps ahead is just how both of us already operate. Nick does it running his gym. I do it building this. Sitting down to formalize the ownership structure wasn't a difficult conversation we forced ourselves to have. It was just the next obvious step for two people who already think that way, so it happened early and easily, without either of us needing to work up the nerve.

So the actual lesson here isn't "have the hard conversation early even though it's uncomfortable," because for me it wasn't, and it isn't "get lucky with who you know" either, since that's not something you can act on. The real move is simpler and anyone can do it: before you pay a stranger or spend a weekend trying to learn something cold, ask the people already in your life first. A co-owner, a parent, a friend from a completely different career, someone you already trust might already have the piece you're missing, or know someone who does, and it costs nothing to ask before you go looking anywhere else. Separately from that: whatever the admin actually is in your case, start it earlier than feels necessary, because every part of it takes longer than the version in your head.

CHAPTER 5

Learning to Speak the Language

Nobody hands you a glossary when you start a business. You just get thrown into rooms, or conversations, or your own spreadsheets, where words like MRR and churn and CAC get used like everyone already knows what they mean, and you nod along doing the mental math of pretending you're not two sentences behind.

A lot of the language I did manage to pick up before I learned it the hard way came from watching people like Alex Hormozi and Dan Martell. I didn't sit through a business degree for any of this. I sat through hours of their videos instead, the kind where they explain a term in plain words instead of hiding behind it, and I'd rewind the specific bit where a word came up that I didn't actually understand yet, not just recognize the sound of. Between the two of them I got more usable business literacy than I think a semester of a formal course would have given me, mostly because they were talking about the actual mechanics of running something, not theory.

MRR is monthly recurring revenue, ARR is that same number stretched out over a year, and churn is the rate you lose paying customers every month. I didn't learn any of those as definitions first. I learned them by having to build financial projections into a pitch deck before I had any real revenue to point to, and realizing partway through that a number on a slide and a number that's actually happened to your business are two completely different things, even when they're written in the same font on the same page. Anyone can

put a growth curve into a deck. The words only become real once you're tracking them against something that's actually landed in an account.

Churn in particular took me longer to respect than it should have. It sounds like a small, dry percentage until you actually sit with what a leak like that does over time, quietly costing you a slice of what you already built every single month whether you're paying attention to it or not, forever, unless new growth keeps outrunning it. I didn't understand that as a feeling until I understood it as an ongoing cost instead of a one time number.

The term that actually humbled me came with a result attached, not a definition. Three hundred and seventeen cold outreach emails sent, trying to get gyms interested in what I was building. One genuine human reply. Not a sale, a reply. That's what customer acquisition cost actually means once you've lived it instead of read about it. It's the moment cold outreach stopped being something I assumed would work because I was putting effort into it, and started being something I actually measured, because the measurement was the only thing that told me the truth.

A few weeks before I sat down to write this chapter, I saw a version of the same lesson happen to somebody else, and it's stayed with me since. I was at the Oz Fitness Expo for two full days, there with Totality, standing around with a mate who was there promoting her own business. Toward the end of the second day we sat down to watch a panel of some genuinely big Australian gym owners talk through the mistakes they'd made and where they were taking things next. It was a good panel, worth the trip on its own. During a break in it, my mate showed me a video on her phone. It was of two young guys who'd apparently had their own stall at that same expo, first year in business by the sound of it, first expo. I'd walked that entire floor for two full days and never once saw them. In the video, by their own account, they'd spent fifty thousand dollars to be there, stock and the stall together, promoting an energy drink. Over the whole weekend

they sold a hundred and fifty dollars of it. Not a hundred and fifty thousand. A hundred and fifty. I don't know either of them, and I'm not telling this to laugh at them, plenty of people spend big before they have the traction to justify it, myself included in smaller ways. I'm including it because it's the clearest version of unchecked CAC I've personally come across. Take from that what you will.

The whole point of writing any of this down is to actually help you, not just to tell war stories at you, so here's the plain list, no story attached, the terms that mattered most to me in the order I actually needed them.

MRR (Monthly Recurring Revenue): the total amount of predictable subscription revenue coming in every month. The core health number for any subscription business.

ARR (Annual Recurring Revenue): your MRR multiplied by twelve. Used for talking about the business at a yearly scale, especially in projections and pitches.

Churn: the percentage of paying customers (or revenue) you lose in a given month. A quiet, compounding leak, not a one time hit.

CAC (Customer Acquisition Cost): what it actually costs you, in money, time, or both, to acquire one paying customer through a given channel. You don't know your real CAC until you've actually run the numbers on something that failed, not just something that worked.

LTV (Lifetime Value): the total revenue you can expect from one customer over the entire time they stay with you. Worth knowing because it's what CAC has to be measured against, spending more to acquire a customer than they'll ever be worth is a losing trade even if the sale itself feels like a win.

Runway: how many months you can keep operating before the money runs out, at your current rate of spending.

Burn rate: how fast you're spending money each month, the number that determines how long your runway actually lasts.

Learn these before you need them if you can. I learned most of mine after I needed them, which works, but it costs more than it has to.

CHAPTER 6

Nobody's Watching Your Work But You

When you work inside a company, there's usually a structure around you that catches what you miss. A code reviewer looks at what you built before it ships. A QA team clicks through the paths you didn't think to test. Someone in security occasionally asks uncomfortable questions about who can see what. None of that exists when it's just you. It doesn't fail to exist, exactly. It's simply not there at all, and the absence doesn't announce itself. Everything just looks fine, right up until it isn't.

The auto-regulation engine is the single feature I'd point to as proof this platform is genuinely different from anything else in this space. A client logs a set, the engine compares it against what was expected, flags the coach with a recommendation, the coach approves it in one tap, and the adjusted programme is supposed to push straight to the client's screen automatically. That loop is the whole pitch. At some point, for no reason except that I'd built up the habit of not trusting something just because it looked right on both ends, I sat down and actually traced that flow from the coach's approval all the way through to what the client's app was supposed to receive. The approval was firing correctly. The client's screen was never actually updating. The entire loop, the one feature I'd been telling people made this platform worth paying for, dead-ended silently in the middle, with no error and nothing that would have told me unless I went looking for it on purpose. Nobody flagged it, because there was nobody positioned to flag it. No QA team running through the flow after every change. No support ticket, because it hadn't reached

enough real users yet to surface on its own. It got caught for one reason only, because I decided to check something that had no obvious reason to be broken.

Somewhere around the same period I ran a full audit on data access across the platform, again for no specific reason, just because row level security is exactly the kind of thing that can be quietly wrong for months without a single symptom. It found that certain functions handling gym rosters would return the complete client list of any gym on the platform to any logged in user, not just their own. Real names, real emails, real phone numbers, visible to an account that had no business seeing them. Alongside that, a permissions gap on the Xero integration and a way to hijack the OAuth connection flow if someone knew to look for it. None of this had a symptom. Nobody had complained. It just sat there, correct looking on the surface, wrong underneath, for exactly as long as nobody happened to check. There's no compliance team assigned to a one person company. There's no security researcher getting paid to poke at it. The only reason any of this got found before it became a real problem for a real gym's members is that checking had become a habit rather than a response to something going wrong.

Even with all of that checking, some things only reveal themselves once real money is actually moving through the system live, not in testing. A Nutrition customer went through checkout, the payment genuinely succeeded, and they ended up locked out of their own account afterward. The cause turned out to be shared customer identifiers in Stripe misrouting events between the two apps, a specific interaction that no amount of careful testing beforehand had surfaced, because it only existed once real customer data crossed both systems at once.

None of this happened through inspection alone, and that's worth separating out clearly. I had automations running full diagnostics across the platform every week, checking things a human wouldn't think to check that often, and on top of that I was doing my own

manual walkthroughs of the app almost every single day. Even with all of that layered together, some things simply can't be found by staring at code or reading through a function line by line. Code can look completely correct and still fail the moment it's actually used, under real conditions, by a real person doing something slightly different from what you pictured while writing it. The auto-regulation loop looked right on both ends until I actually walked a recommendation through it start to finish. The Stripe routing looked right until two real customers' data crossed paths in a way no test data had ever recreated. Looking is necessary. It is not sufficient.

I don't tell you any of this to make a case for how careful I am. The actual point is closer to the opposite. Every one of these could have sat broken indefinitely, silently costing trust or data or money, and the only thing standing between silently broken and found and fixed was a habit, not a talent. Use your own product constantly, more than feels necessary. Get real people using it too, even if that means giving it away for free at first just to get more hands on it than yours alone. Try to break every part of it yourself before a paying customer ever gets the chance to. Things will still break anyway, they always do, that part never fully stops. But the difference between doing this work first and skipping it is the difference between a small, cheap failure and a costly one landing on someone who actually paid you.

CHAPTER 7

Automations, and Filtering the Advice

Once there was an actual product, a whole second job appeared underneath it, the job of running the business itself, and I went looking for frameworks to help with that the same way I'd gone looking for market numbers back at the start. Dan Martell has a full operating system for exactly this, built around turning different parts of running a company into AI agents. I sat down with it properly, ready to apply the whole thing.

The first real decision wasn't which piece to build. It was admitting how much of it didn't apply to me at all yet. That playbook is written for operators scaling past ten million dollars, with departments and actual headcount underneath them. What I actually had at the time was two owners, no employees, around twenty seven total user profiles across both apps, six support tickets in the entire history of the product, and two hundred and forty one leads sitting in a cold outreach pipeline with zero conversions yet. Half the framework, hiring AI, recruiting bots, engineering review tools, dedicated finance software, assumes a team and a transaction volume that simply didn't exist for me yet. Running the whole system anyway because the material told me to would have been busywork dressed up as progress, and I think that specific trap catches a lot of people who read one good framework and try to become the company the framework was written for instead of the company they actually are.

What I did have running already, once I looked honestly at it, was more than I expected, and each one earned its name by actually having one specific job, not just existing because automation sounded

good. Totality Reports runs four days a week without me touching it, a different topic each day, nutrition on Wednesday, training on Thursday, mindset on Friday, quick tips on Saturday, feeding people useful content on a schedule instead of whenever I remembered to post something. Totality Weekly Cold Outreach has a completely different job, finding real gyms and PT businesses, sending a five stage sequence of emails out to them over several weeks, tracking every lead through that sequence, and reporting back to me on its own after every run. The third piece is the Command Center, and its job is different enough from the other two that it's worth its own space here.

The Command Center is the one I want to spend more time on, because it doesn't just report on what already happened, it holds decisions I made in advance for money I don't have yet. Day to day it tracks the obvious things, actual ad spend against actual return, what's going out in tools and subscriptions every month, where real revenue sits compared to where I said it would be. But underneath that live tracking sits something else entirely, a full plan for exactly how every dollar gets split, between advertising, my own salary, dividends to Nick and me, and money set aside in reserve, at every revenue milestone from twenty five hundred dollars a month all the way up to one million dollars a month. Twenty four separate steps, each one with its own numbers already decided.

I'll say the obvious thing before you do. Planning exactly how a business making a million dollars a month will split its money, when the business currently makes nowhere near that, is extreme, and none of it needed to exist yet for anything to keep working today. But building it taught me something I don't think I'd have learned any other way, which is that most founders make their biggest financial decisions for the first time under pressure, with real money already on the table and emotions already involved. I decided in advance, calmly, with nothing at stake yet, that business savings should never drop below fifty two and a half percent of revenue no matter how tempting it gets to pay myself more, that ad spend is a ceiling to scale

into once the return actually proves itself rather than a target to hit because a number felt exciting, and that founder pay caps out at a fixed amount even once the business is doing extremely well, so growth keeps feeding the business instead of just feeding us.

The one exception I built into the entire plan tells you more about why I did this than any of the actual numbers do. At ten thousand dollars a month in revenue, the plan says that's the point I go full time on this, leaving whatever else I'm doing for income behind. Every other milestone protects that fifty two percent savings floor without exception. That one doesn't, because at exactly that moment my own ability to survive matters more than the business's reserve for one stretch, so I built in a fixed minimum income for myself right there, even though it means the business keeps less than it normally would at every other step. I decided that trade-off on a calm afternoon with a spreadsheet open, instead of discovering under pressure, the week I actually hit ten thousand dollars a month, that I hadn't thought about how I'd actually pay my own bills once I quit.

So yes, it's extreme, and no, a plan for a million dollars a month wasn't remotely necessary the week I built it. But the value of it was never really about having the right number for a milestone I'm nowhere near yet. It's a sign of a habit worth having regardless of the specific numbers: think forward, always, even further than feels useful, because the decisions you make with nothing on the line are almost always better than the ones you're forced to make once something finally is.

The thing I decided to build next, ranked above everything else in the framework, was something for triaging my own inbox and protecting my own calendar, because that was the one category with genuinely nothing built yet and the one costing me the most scattered attention day to day. Underneath that, three more things worth building, in order: something that pulls real language from actual customer support tickets and reviews to use in ad copy and posts, because I have a hard rule for myself about never inventing a stat or a

testimonial, and this does the opposite, it goes and finds the real words instead. Something that takes a raw DM conversation and pulls out the objection, the budget, the next step, so it's usable instead of just sitting in my messages. And a single page weekly scorecard, outreach numbers, new signups, trial conversions, revenue, because you can't actually tell if any of this is working without looking at the same numbers on the same schedule instead of checking in on a feeling.

Just as important as what I decided to build is what I decided not to touch yet. A system for flagging customers at risk of leaving, a real and useful idea, but with six support tickets total and twenty seven users, there was no volume yet to make that worth automating instead of just noticing myself. Hiring or recruiting tools, with no team to hire for. A whole separate prompting system layered on top of what I already had, when what I already had was already doing the same job. Saying not yet to all of that wasn't caution for its own sake. It was matching the tool to the actual size of the problem in front of me instead of the size of problem the framework assumed I had.

The part of listening to people like Martell that actually mattered wasn't following every instruction. It was one small framework underneath all of it: give whatever you're building one specific aim, give it its own narrow identity, give it only the access it actually needs, keep it to one job instead of folding it into something broader, and only widen what it's allowed to do once it's proven itself at the smaller job. That's exactly why Totality Reports, Totality Weekly Cold Outreach, and the Command Center work, each one has exactly one job and stays inside it, instead of one bloated system trying to be all three at once.

It also kept my spending honest. It would have been easy to buy a separate piece of software for every category that framework named, an inbox tool, a workflow tool, a CRM tool, a content tool, a finance tool, and blow well past what I'd actually budgeted for running this thing month to month. Almost all of it was work that Claude and

Supabase could already do directly, once I'd actually scoped the job properly instead of assuming a category needed its own paid product just because it had its own name in someone else's slide.

One more practical thing worth mentioning while I'm on the subject of tools, since it didn't exist in this form when I started and would have saved me real hours if it had. Claude now works through something called Cowork, and one of the things it can do from there is drive your actual computer, including your own Chrome browser, on your behalf. Research that used to mean me manually opening a dozen tabs, or filling out the same kind of form across several different sites, can just get handed over and done in the background while I work on whatever actually needs to be me. It doesn't touch the deciding, that rule from Chapter 3 still holds completely, but for the browsing and clicking and copying that quietly eats an afternoon for no good reason, it's worth knowing it exists.

So if there's a lesson in this chapter worth taking further than my specific tools, it's not "go copy Dan Martell's system" or "go build a Gatekeeper agent." It's that listening to someone smarter than you about this stuff doesn't mean adopting everything they say. It means running their framework against your actual numbers, honestly, and being willing to say not yet about most of it, out loud, even when the person teaching you clearly thinks you should be doing all of it right now.

CHAPTER 8

Eighty Hours a Week, on Purpose

For all of this, I had a full time job as a personal trainer during the day and a second, unpaid job building this thing at night. Nobody was clocking my hours. Nobody needed to see a status update by Friday. The only deadlines in my life were the ones I'd made up myself, which sounds like it should be easier than working for someone else and is actually much harder, because a deadline with no external enforcement only survives if you enforce it on yourself, every single week, with nobody checking.

That's the actual reason I built a Friday check-in for myself instead of just trying to power through on willpower. Every week, the same rubric, the same non-negotiables across every part of my life, not just the business, scored honestly against what I actually did instead of what I meant to do. I'm not describing this because grading yourself weekly is some kind of secret productivity trick. I'm describing it because without some structure forcing me to look at the gap between what I said I'd do and what I actually did, eighty hour weeks quietly become sixty hour weeks that feel like eighty, and you don't notice the slide until months of it have gone by.

It went further than the weekly check-in, too. I asked Nick directly to do something smaller and more specific, every time he saw me in person or called me, ask how my sleep had been, and if it was evening when he called, tell me to go to bed instead of staying on the phone. That's not the kind of accountability that shows up in a business plan anywhere. It's just someone else holding onto the one thing I knew I'd sacrifice first given the chance, sleep, because I could

talk myself out of protecting it on my own but I couldn't talk my way out of Nick actually asking, out loud, every time. Sometimes the accountability you need isn't a system at all. It's telling one specific person the one specific thing you know you'll let slide, and asking them to keep bringing it up until you can't ignore it anymore.

The accountability piece matters more than the hours do. Anyone can grind for a weekend. Sustaining a pace like that for months, around a job that already has its own demands, needs something external checking in on you, because your own judgment about your own effort is exactly the thing that gets worst precisely when you're most tired. I built the Friday check-in and asked Nick for the sleep thing for the same underlying reason, I know myself well enough to know I'd lie to myself about both otherwise, not out of dishonesty, just out of the ordinary human instinct to round your own effort up and your own exhaustion down when you're deep in something that matters to you.

None of this is a case for working eighty hours a week as some kind of badge of honor. I'd tell you plainly, if you can build the thing you're building without doing that to yourself, do it that way instead. I didn't have that option, because I was building something I'd promised a real person, on a deadline I'd set myself, on top of a job I still needed to pay rent. Given those constraints, the actual choice wasn't between eighty hours and a saner number. It was between eighty hours done with some structure around them, and eighty hours done without any, running purely on adrenaline until it ran out.

There's a part of this I haven't said yet, and it belongs here more than anywhere else in the book. Building something like this on the side of a full time job is lonely in a specific way that's hard to explain to someone who hasn't done it. Not lonely because you're alone in the room, most of the actual building happened alone in a room, but lonely in the sense that almost nobody around you is living inside the thing you're building the way you are. I was lucky. Nick believed in it from that first meeting and never really stopped. My family and my

friends stayed around through a year that must have looked, from the outside, like I'd half disappeared into a laptop. I don't take any of that for granted, and I want to be honest that not everyone gets it. I've had people in my own life who weren't interested, who didn't believe in it, who I stopped bringing it up around because that was easier than being met with nothing. That's real too, and if you're finding that out yourself right now, I won't pretend it doesn't sting. If you don't have people around you who believe in you, or who are at least willing to be curious about what you're doing, that's worth taking seriously rather than quietly absorbing. Go find them. They exist. Change your circle if you have to. You don't need a crowd cheering you on, I didn't have that either, but you need more than nobody.

I said in the first chapter that I gave myself eighteen months and beat it by more than a year, in five. I want to end on that number one more time, because I think it's the whole book in miniature. It wasn't luck, and it wasn't just talent either. It was a real problem I'd actually seen with my own hands, a deadline with money and trust attached to it, a business structure I didn't understand but found people who did, a habit of checking my own work because nobody else was going to, and a weekly system, plus one friend asking a simple question at the right moments, that kept me honest even when nobody else would have noticed if I'd quietly let it slide.

If you take one thing from this whole book, take this. None of what I did required a rare kind of genius. It required deciding, over and over, on ordinary unremarkable days when nobody was watching, to actually do the thing I'd told myself and one other person I was going to do. That's available to you too, whatever you're building, whatever it costs you. It cost me a lot of nights and a lot of weekends. It's also the reason this book exists at all instead of just being an idea I mentioned to my boss once and never followed up on.

CHAPTER 9

The Checklist

Everything else in this book was the story. This is the part you come back to.

The lessons, one per chapter

1. Pay attention to patterns in your own actions, not just to what people say back to you. The real problem usually shows up while you're already doing something else.
2. Audit what your idea actually needs against what already exists, feature by feature, until the gap is a list instead of a feeling.
3. Don't think in bootstrap versus investment. Attach a real, checkable deadline to any money you take, including from yourself.
4. You don't know what you don't know about running a business. Ask the people already in your life before hiring a stranger cold.
5. The vocabulary comes after the mistake, not before it. Learn it from people who explain it plainly, and measure your real results instead of trusting your effort.
6. Nobody is checking your work but you. Build the habit of trying to break your own product before a paying customer does.
7. Filter every framework you're taught against your actual numbers. Give each automation one job. Plan further ahead than feels useful, calmly, before you have to decide anything under pressure.

8. Eighty hour weeks need structure and outside accountability around them, or they quietly become something worse than eighty hours while feeling the same.

Tools worth knowing about

- Claude, for the actual building, database work, and code, once you already know exactly what you want and why. It executes. It doesn't decide.
- Claude Cowork, specifically for handing off browser busywork, research, forms, repetitive lookups, so your own time goes to deciding, not clicking.
- Supabase, for the backend and database underneath an app like this.
- Stripe, for handling payments and billing, whether that's subscriptions coming in or paying other people out.
- Vercel and Expo, for hosting and building, both usable on a free tier for a long time before you need to pay for anything.
- HeyGen, kept strictly for ad and marketing video, never for anything inside the product itself.
- A scheduled task tool of some kind, for anything that needs to run on a schedule without you remembering to trigger it.
- Claude also connects to thousands of other tools and services beyond the ones listed here. If you're not sure which ones would actually help your specific business, just ask Claude directly, it can tell you.
- You can create Projects inside Claude and let it build up memory about your business over time, so you're not re-explaining the same context every single conversation.

Who to actually watch

- Alex Hormozi and Dan Martell, for plain language explanations of the actual mechanics of running a business, not theory dressed up as content.
- Beyond that, watch fewer people more closely rather than more people shallowly. You'll get more usable language from a handful of sources you actually rewind and study than from an endless feed of business content.

What to learn, no matter what you're building

- MRR, ARR, churn, CAC, LTV, runway, and burn rate, at minimum. Go back to Chapter 5 for the plain definitions.
- Enough about company structure, directorship, and trusts to know what questions to ask your accountant, even if you never become an expert in any of it yourself.
- What your own product's biggest security and data-access risks actually are, specifically, not generally.

Pitfalls to avoid

- Pitching the smaller, safer version of your idea because it's easier to say out loud.
- Building something wider than deep, one shallow feature copied across five categories, instead of one real problem solved completely.
- Adding a buzzword feature because it's expected, instead of the honest, explainable version that actually works.
- Taking money, from anyone, with no real condition attached to it.
- Assuming an ABN or its equivalent is the whole checklist. It almost never is.
- Skipping a shareholder or ownership agreement because things are going well and the conversation feels unnecessary.

- Trusting that code which looks correct is correct. It isn't, until it's actually been used.
- Copying a business framework wholesale instead of checking it against your real numbers first.
- Treating eighty hour weeks as proof of anything, on their own, without the structure and the outside accountability that make them sustainable instead of just exhausting.

Take care of yourself, too

- Keep exercising. It's usually the first thing to slip, and the first thing you'll regret dropping.
- Pay attention to your own mental health and wellbeing, not just the business's numbers.
- Protect your sleep on purpose. Don't just let it be whatever's left over at the end of the day.
- Keep people around you that you can talk to who have nothing to do with work, and nothing to do with how you're feeling either. Just friends.
- Have something you genuinely enjoy that's got nothing to do with the business, something that actually lets you switch off.
- Know your reason for not wanting to switch off, clearly enough to say it in one sentence. It's what gets you through the stretches when everything else runs out.

Thank You

This book exists because a few people believed in something before there was anything yet to point to. Nick, for the fifteen minutes that turned into a hundred, and for everything since. My family, for staying patient through a year of missed calls and half-finished conversations while I was somewhere else in my head, building. My friends, for still being there on the other side of it. And everyone else who looked at an idea with no proof behind it yet and decided to believe it anyway.

Thank you for that.